Lớp: **IT003.R17**

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: Trần Võ Nhật Nam - 24521123

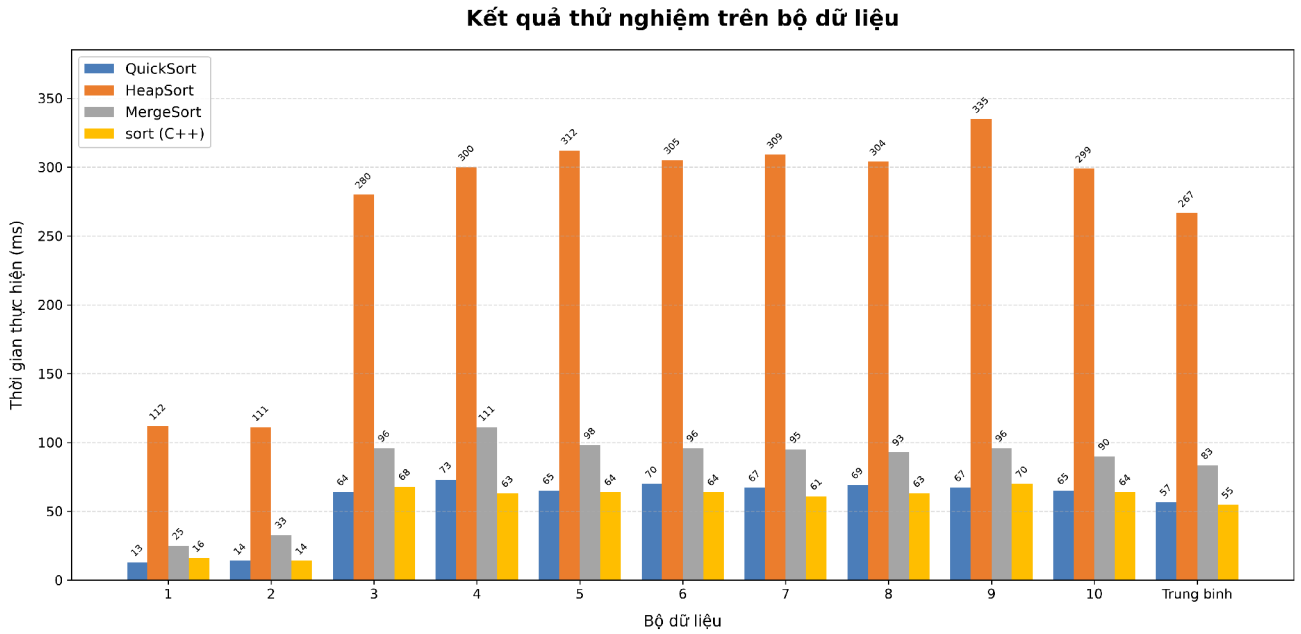
Nội dung báo cáo:

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện

Dữ liệu	Thời gian thực hiện (ms)			
	Quicksort	**Heapsort**	**Mergesort**	**sort (C++)**
1	13	112	25	16
2	14	111	33	14
3	64	280	96	68
4	73	300	111	63
5	65	312	98	64
6	70	305	96	64
7	67	309	95	61
8	69	304	93	63
9	67	335	96	70
10	65	299	90	64
Trung bình	**56.7**	**266.7**	**83.3**	**54.7**

2. Biểu đồ cột thời gian thực hiện



II. Kết luận:

1. Thuật toán QuickSort:

• Độ phức tạp: Trung bình là O(N log N), trường hợp xấu nhất là O(N^2).

• Kết quả trên dãy 1 (Tăng dần - 13.0 ms) và dãy 2 (Giảm dần - 14.0 ms): Khi cài đặt QuickSort theo bài giảng, chọn pivot là phần tử chính giữa mảng pivot = a[(left + right) / 2], trên mảng đã có thứ tự tăng dần hoặc giảm dần, phần tử chính giữa này chính là giá trị trung vị của dãy. Nhờ đó mảng được chia đôi rất cân bằng, không bị rơi vào trường hợp xấu nhất O(N^2) và không bị tràn ngăn xếp (Stack Overflow). Do các phần tử đã gần đúng vị trí nên số lần hoán đổi ít, giúp QuickSort chạy cực nhanh.

• Trên 8 dãy ngẫu nhiên: Thời gian dao động từ 64 ms đến 73 ms (trung bình 56.7 ms). Thuật toán hoạt động rất hiệu quả, tốc độ tương đương với hàm sort của C++.

2. Thuật toán HeapSort:

• Độ phức tạp: Luôn là O(N log N) trong mọi trường hợp, không dùng bộ nhớ phụ (O(1)).

• Hiệu năng thực tế: Chạy chậm nhất trong cả 4 thuật toán, trung bình mất 266.7 ms.

• Giải thích nguyên nhân: Cấu trúc cây Heap lưu trên mảng khiến các bước truy cập phần tử cha - con bị nhảy cách quãng xa (i -> 2i+1, 2i+2). Với mảng lớn 1 triệu phần tử, việc này gây ra nhiều hiện tượng Cache Miss (bộ nhớ đệm CPU không chứa sẵn dữ liệu), CPU phải tốn thời gian lấy dữ liệu từ RAM nên tốc độ thực tế bị chậm hơn.

3. Thuật toán MergeSort:

• Độ phức tạp: Luôn là O(N log N) trong mọi trường hợp, có tính ổn định (Stable Sort).

• Hiệu năng thực tế: Thời gian trung bình là 83.3 ms, đứng thứ 3 sau sort C++ và QuickSort. Trên dãy tăng dần (25.0 ms) và giảm dần (33.0 ms), MergeSort chạy nhanh hơn mảng ngẫu nhiên (khoảng 90 - 111 ms).

• Đặc điểm: MergeSort cần mảng tạm kích thước O(N) để trộn hai nửa mảng lại với nhau.

4. Hàm sort của C++ (std::sort):

• Hiệu năng thực tế: Chạy nhanh nhất tổng thể với thời gian trung bình 54.7 ms.

• Giải thích: Hàm sort trong thư viện <algorithm> của C++ sử dụng thuật toán Introsort (kết hợp thông minh giữa QuickSort, HeapSort khi đệ quy quá sâu để tránh O(N^2), và InsertionSort cho các đoạn mảng nhỏ). Ngoài ra, trình biên dịch C++ tối ưu hóa mã nguồn rất tốt nên đạt hiệu năng cao nhất.

=> KẾT LUẬN

1. Hàm sort của C++ là lựa chọn tốt nhất và tiện lợi nhất khi làm việc thực tế nhờ được tối ưu toàn diện.
2. QuickSort nếu chọn pivot hợp lý (ở giữa mảng) là thuật toán tự cài đặt nhanh nhất trong các thuật toán thông dụng.
3. MergeSort chạy ổn định, phù hợp khi cần bảo toàn trật tự ban đầu của các phần tử bằng nhau.
4. HeapSort đảm bảo O(N log N) lý thuyết và tiết kiệm bộ nhớ, nhưng thực tế chạy chậm hơn trên mảng lớn do hạn chế về truy xuất bộ nhớ.

III. Thông tin chi tiết – link github, trong repo gibub cần có

1. Báo cáo
2. Mã nguồn
3. Dữ liệu thử nghiệm

- Link github : NhatNam1610/IT003_Sorting_Benchmark: Bài tập Cấu trúc dữ liệu và giải thuật